LISTENING POST · TECHNICAL REPORT LLNV-TR-2026-02 · PREPRINT

Conspicuous Compilation: Raising the Wall-Clock Cost of Automated Reverse Engineering

Three cooperating methods, their attacker cost and their defender cost, measured in Rust

William Spicer Berry[1] · Seth Steele[1] · Daniel Steele[1]

1 Listening Post, Inc. · {wberry, ssteele, dsteele}@listeningpost.ai

Preprint v1 · October 26, 2026

ABSTRACT

Automated, agent-driven reverse engineering has wreaked havoc on the industry, taking away whatever privacy a shipped binary used to have. We study *conspicuous compilation*: emitting a behaviourally identical but deliberately analysis-hostile binary, one made hard to understand for agents and humans alike. Our goal was to raise the wall-clock time an automated decompiler must spend to understand a compiled program. We combine three methods: control-flow flattening, opaque-predicate and mixed-boolean-arithmetic (MBA) hardening, and runtime-selected diverse variants. We measure each alone and together across sixteen Rust functions, confirm with an XOR fingerprint that every build computes the same result, and run the statistics only on runs that pass that check. In our measurements decompile cost rose to several hundred times the plain build. We write as industry professionals working with artificial intelligence, approaching this as a practical problem rather than from a dedicated research program. We are optimistic that compilation obfuscation like this can help against agents, but more research, innovation, and attention is needed before it is usable in production systems.

KEYWORDS

software protection · control-flow flattening · opaque predicates · mixed boolean arithmetic · runtime diversity · decompilation cost

1 The goal, and the term

Reverse engineering has always been possible; what changed is its price, because decompilers driven by language-model agents make scalable, automated analysis cheap. We respond with a compiler that deliberately inflates the cost of automated analysis while preserving behaviour.

We call it **conspicuous compilation** because the binary itself is what gets obfuscated, and obviously so: the source stays readable and the semantics are unchanged, and only the binary is made expensive to read. The question we ask is whether, done this way, we can make a binary meaningfully more expensive to decompile with a tool like Ghidra.

2 The three methods

Method A, control-flow flattening. Each function becomes a dispatcher loop whose state variable carries execution order [1, 7, 8], surrounded by `J` bogus states that never execute (Figure 1).

Method B, opaque predicates and MBA. Every transition routes through an always-true opaque predicate [2] (`7a²−1 ≠ b²`) and the state constants are mixed-boolean-arithmetic masked [5], both behind an optimizer barrier, so the compiler cannot prune the dead edges. We cite prior work using methods similar to Method B that we found after independently reaching the same ground through agent-driven mathematical exploration. We cite it because it is fair to assume their work is in the training data that led us to our result, and we thank them for it.

Method C, runtime-selected diverse variants. This one is the simplest. We emit `K` independent, shuffled copies of the function and use a hash of the current timestamp to pick which one runs on each call.

The methods cooperate rather than stack independently: flattening alone is deleted by the optimizer, and diverse variants only stay distinct once hardened, so method B is what makes methods A and C survive compilation (Figure 3). Flattening and opaque predicates have been combined before at the IR level [4]; our contribution is the controlled measurement of how they compose and what each costs.

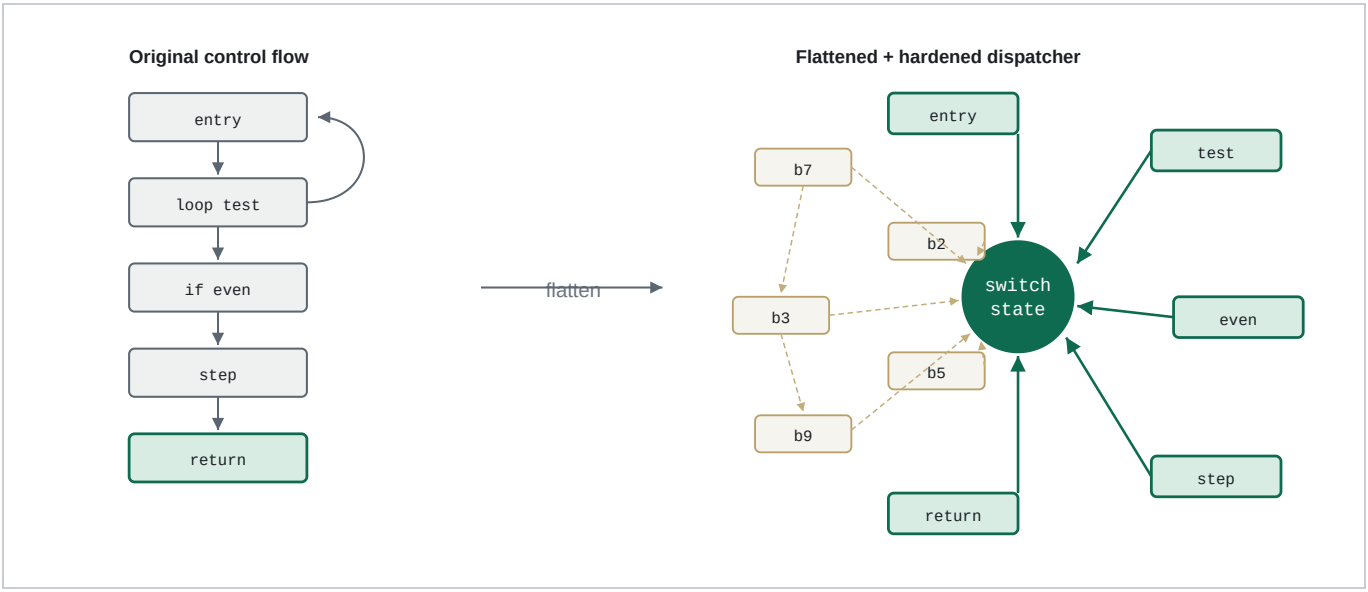


Figure 1. Method A. Flattening turns a linear control-flow graph (left) into a dispatcher star (right); green nodes are real segments, tan dashed nodes are bogus states. The dial `jumps = J` sets how many.

Naive case

plain even_fib_sum, 63 instructions, one readable loop

```
even_fib_sum:
mov $0x1,%ecx # a = 1
mov $0x2,%r8d # b = 2
xor %eax,%eax # sum = 0
jmp .test
.step:
mov %rcx,%r8
add %rsi,%r8 # t = a + b
mov %rsi,%rcx
cmp %rdx,%rsi
ja .done
.test:
mov %r8,%rsi
test $0x1,%cl # even?
jne .step
add %rcx,%rax # sum += a
mov %rax,0x10(%rdi)
jmp .step
.done:
mov %r8,0x8(%rdi)
ret
```

The loop is right there to read.

Methods A + B (flattened + hardened)

same function, 3,890 instructions, the loop dissolved

```
even_fib_sum:
movq $0x334922b2,-0x10(%rsp) # masked state
mov -0x10(%rsp),%edx
mov %edx,%ecx
not %ecx
imul %edx,%ecx # MBA, always 0
and $0x1,%ecx
or $0x1849b6,%rcx # real const
movq $0xbbf83d4,-0x10(%rsp)
mov -0x10(%rsp),%edx
not %esi
imul %edx,%esi # MBA, always 0
and $0x1,%esi
... ~130 dispatcher states ...
xor %rsi,%r9
imul %rax,%rax # a²
imul $0x7,%rax,%rcx # 7a²
dec %rcx # 7a²−1
cmp %rcx,%r8 # ≠ b² (opaque)
jne .state_7b3f
... 3,800+ more ...
```

Same answer. Nothing left to read.

Figure 2. The same function, disassembled. Left: the plain build is a readable loop in 63 instructions. Right: methods A and B produce 3,890 instructions in which the loop has dissolved into a dispatcher over MBA-masked state guarded by the opaque predicate, computing the identical result.

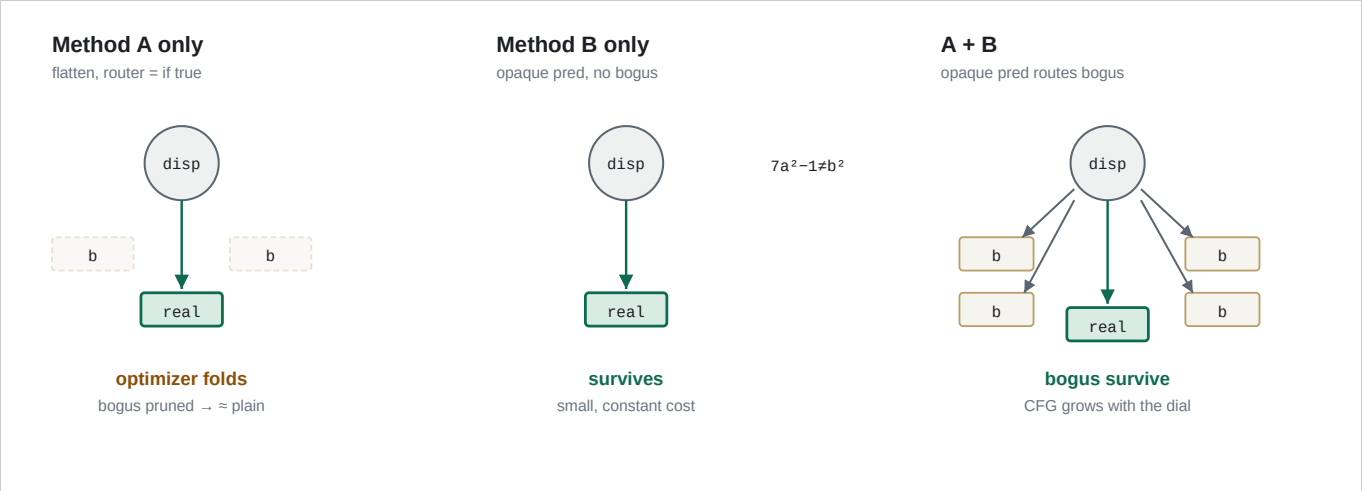


Figure 3. Why they cooperate. Method A alone is pruned by the optimizer; method B alone guards only the real path; together the bogus states survive and the graph grows with the dial.

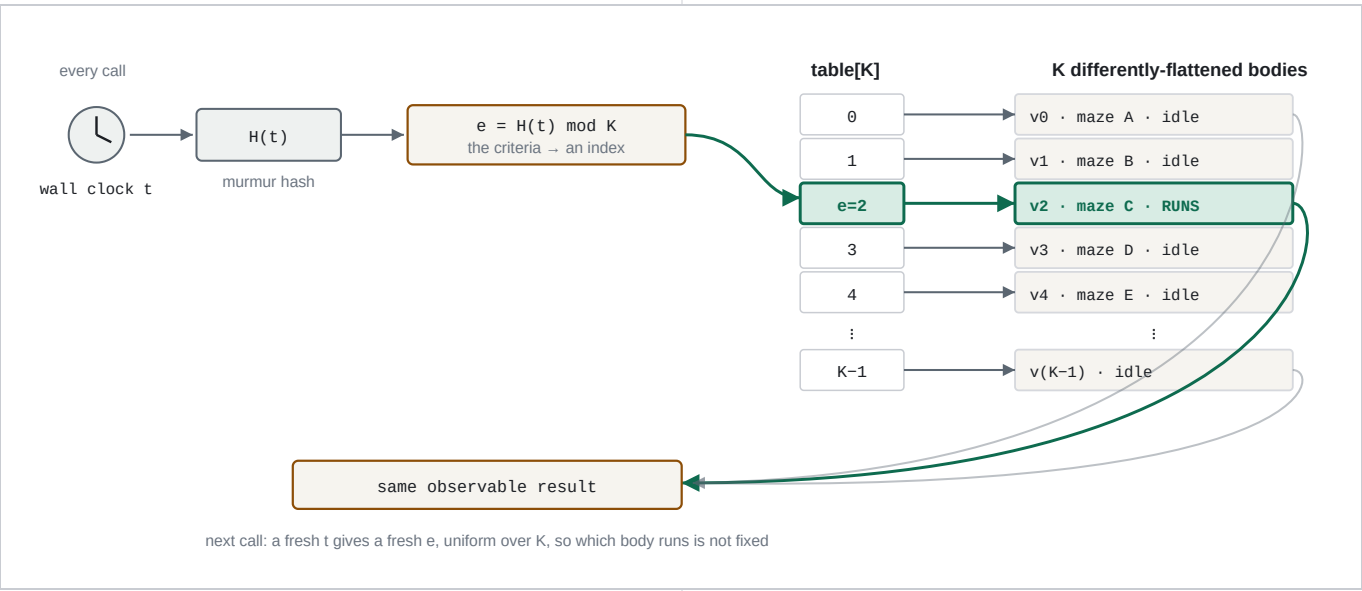


Figure 4. Method C. A wall-clock reading is hashed to an index `e = H(t) mod K` that indexes a table of K differently-flattened bodies and runs one of them; all K compute the identical result, and e changes from call to call, so which body runs is not statically fixed. The K bodies differ in their constants and also in size and layout: distinct bogus-state counts, dead-edge wiring, and arm order.

3 Experimental design

The study is done in Rust, whose optimizer keeps the annotated functions as standalone symbols, giving a clean same-function baseline in which the plain and obfuscated builds differ only by the transformation. Each condition builds the same sixteen small functions, each with a different shape of control flow. Every function returns a number, and the build folds all sixteen into one fingerprint. That fingerprint comes out the same for the plain build and for every obfuscated build, so each obfuscated binary computes exactly the same answers as the plain one. The attacker we model points an automated decompiler at the binary; in the limiting case a language-model agent is instructed to “load the binary in Ghidra, decompile every function, and summarise what each one does.” To measure that cost deterministically and reproducibly, we remove the agent and drive Ghidra 12.1.4’s [6] headless decompiler directly, calling it on each corpus function ten times and recording the median wall-clock; an agent’s own reasoning time sits on top of this floor. Defender cost is runtime nanoseconds per call. The machine the results were gathered on was pinned to performance mode and on AC power. All compilation and measurement ran on one machine: an AMD Ryzen 7 7840HS (8 cores / 16 threads, up to 5.14 GHz, 16 MiB L3), 44 GB RAM, Ubuntu 24.04.5 (Linux 7.0.0), with rustc 1.94.1 and Ghidra 12.1.4 on OpenJDK 21.

4 Attacker cost: decompile wall-clock

Three complementary sweeps all raise decompile time steeply. Depth cranks one function's dial; breadth increases how many functions are obfuscated; variants increases how many diverse bodies each function carries. Method A alone adds nothing (its bogus states are pruned, binary size identical to plain); method B alone roughly doubles cost; combined they are super-additive, 52× where independence predicts 2.0×. The variants sweep keeps methods A and B on but pinned to their floor (`jumps=8, mba_depth=1`) and turns only `K`, so its 704× at `K=64` is the cost of all three methods together with A and B held minimal—not of method C in isolation.

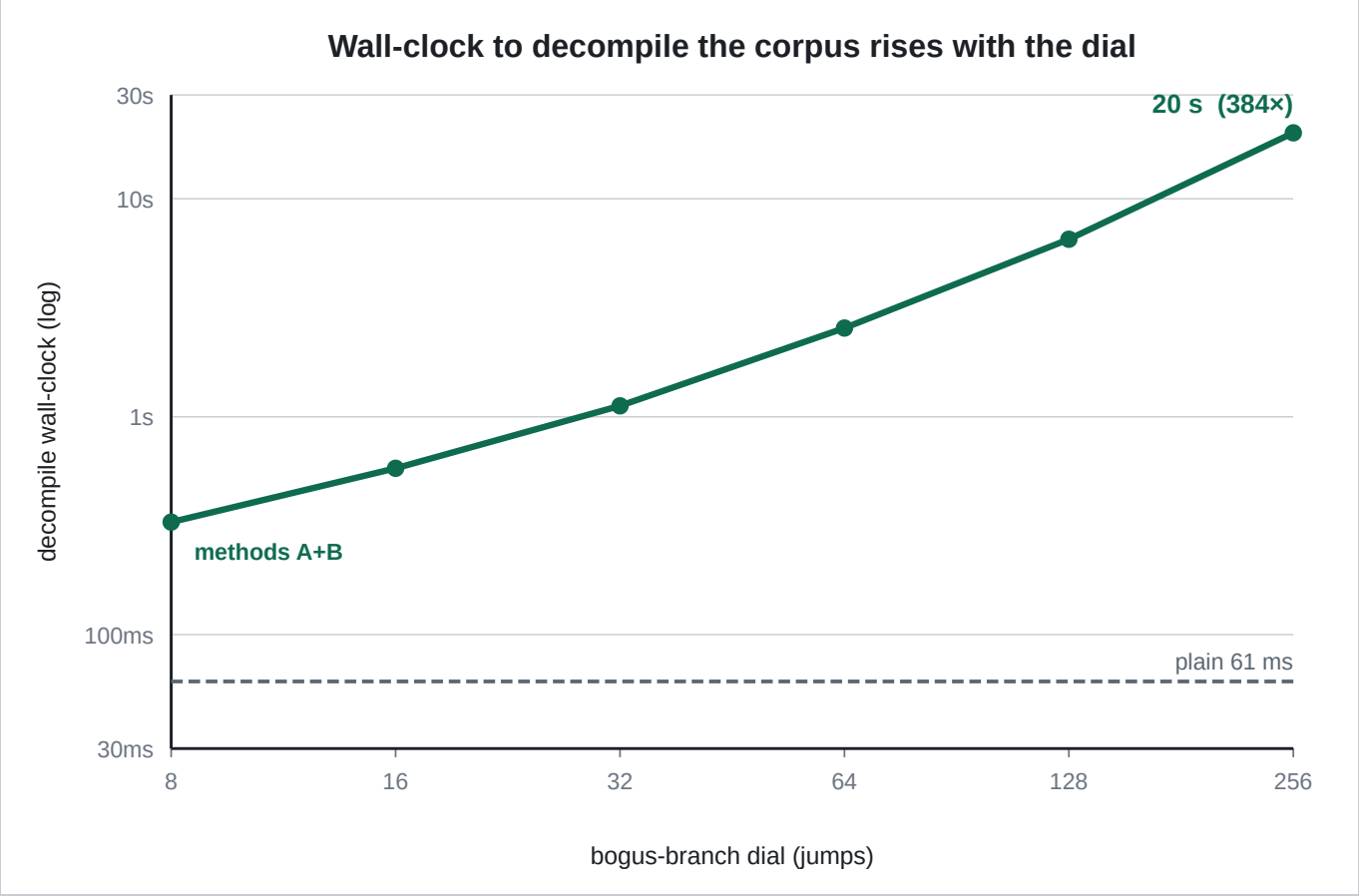
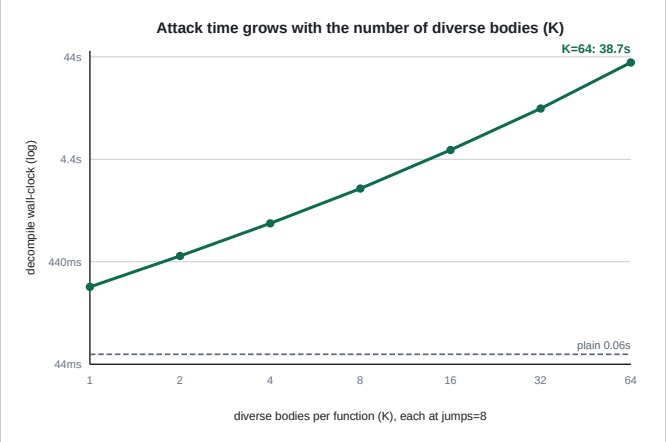


Figure 5. Depth sweep. Decompile wall-clock for the corpus rises from 61 ms (plain) to 20 s (384×) as the bogus-branch dial is turned up, for identical behaviour.



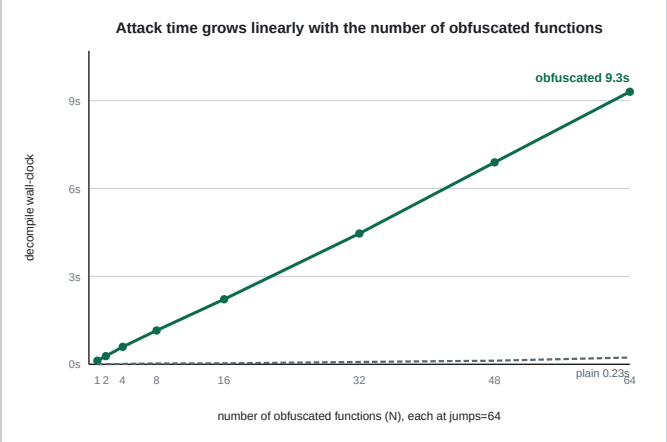


Figure 6 (left). Variants: decompile cost grows with K diverse bodies, to 704× at K=64. **Figure 7 (right).** Breadth: cost grows linearly with the number of obfuscated functions, about 0.15 s per added function.

Table 1. Attacker decompile cost vs plain, three sweeps. Correctness gate held for every row.

SWEEP	KNOB	DECOMPILE COST
Depth (jumps)	8–256	**6.3× → 384×**
Variants (K)	1–64	**4.6× → 704×**
Breadth (funcs)	1–64	linear, ~0.15 s / func

5 Defender cost: runtime

Runtime is median added nanoseconds per call versus plain, robust to one microsecond-scale function that would dominate a raw average. Methods A and B add only a few nanoseconds per call, two orders of magnitude below method C, but they are not quite flat or free. Method A alone measures zero because the optimizer deletes the flattening; once method B keeps it from being optimized away, the real path runs through the surviving dispatcher, so A+B rises from about 2 to 5 ns across the dial. Bogus code still never executes; the cost is the few extra operations the real path now carries. Method C dominates defender cost, because it does work on every call: a wall-clock read plus an unpredictable dispatch among K bodies.

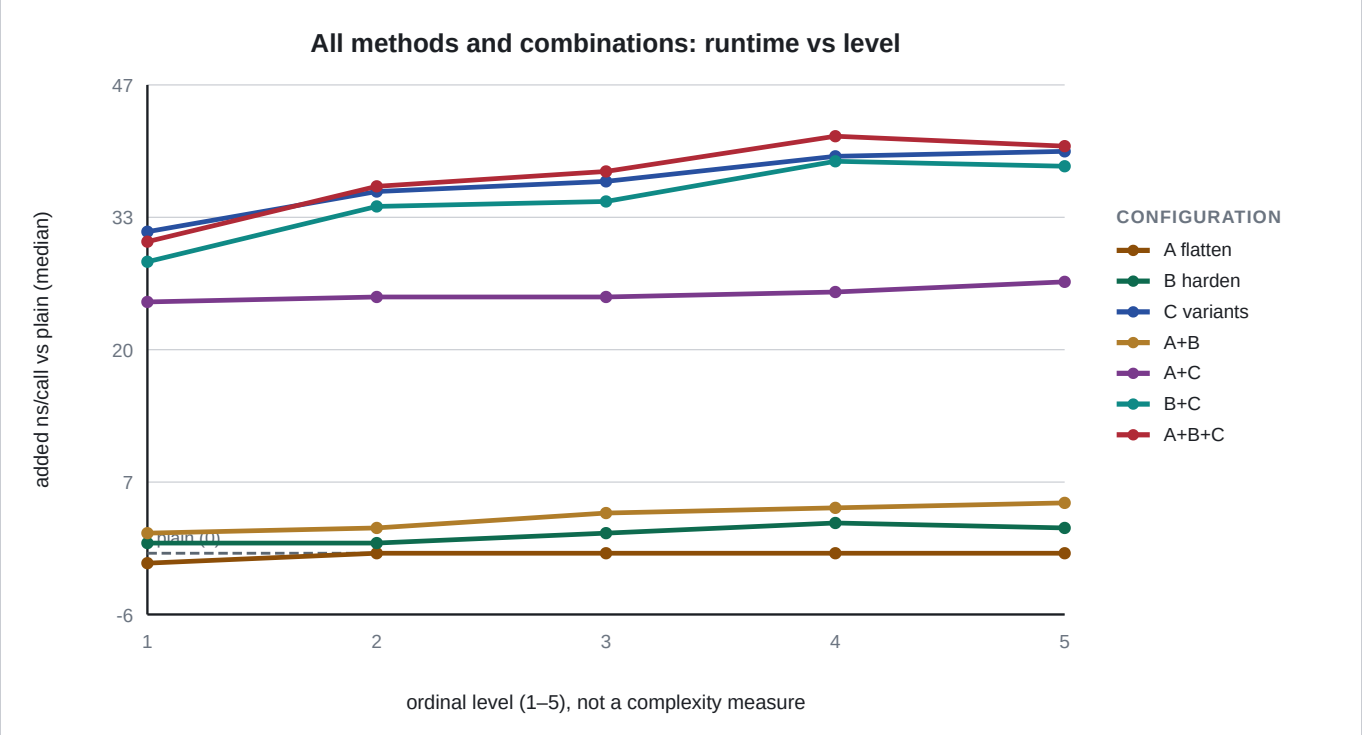


Figure 8. Runtime by method and combination, as added ns/call vs plain. The horizontal axis is an ordinal *level* (1–5), not a calibrated measure of complexity: at each level every method is turned to the next notch of its own knob, which lets the curves share one axis even though a notch is not equivalent across methods. Level sets method A's `jumps` = 8/16/32/64/128, method B's `mba_depth` = 1/2/3/4/5, and method C's `variants` = 2/3/4/6/8; a configuration leaves the knobs it does not use at their floor. Read each curve for its own trend, not as a method-to-method comparison at a fixed level. Everything without method C clusters near zero; everything with it sits at +25 to +42 ns.

Table 2. Added ns/call at the hardest level tested. Median over the corpus.

CONFIGURATION	ADDED NS/CALL
Method A (flatten)	~0
Method B (harden)	+2.5
Methods A+B	+5
Method C (variants)	**+40**
All three	**+40.5**

Attacker pays on three knobs; defender mostly on one

Attacker decompile cost scales with every knob; defender runtime cost is dominated by method C, with methods A and B adding only a few ns that grows slightly with their dial. Crank the static methods hard for a few nanoseconds; add runtime diversity only as far as the per-call cost allows.

6 Threat model and limitations

We model a **static, automated** attacker: a decompiler, possibly agent-driven. We did not consider any other avenue of attack. A **dynamic** attacker who runs the binary under a debugger, for instance, sees the one real path directly, and method C's

diversity resists trace-diffing but not a debugger. This study is simply a measure of economic expense against a named tool in a specific category [3], and we make no guarantee that agents won't find ways around it that we did not foresee. We did try to break it ourselves a bit before publishing and did not turn up anything clever enough to mention. The super-linear growth is an empirical fit, not a proved bound.

We prove no bound on recovery; future agents or decompilers may learn to undo these transforms, which we do not pursue here. Our question was only whether cheap, behaviour-preserving transforms make common automated analysis (an LLM agent or a stock Ghidra pass) measurably slower.

7 What we hope is true

This section is conjecture, and each claim below needs empirical validation in further review. The scenario is server-side software under a modern continuous-integration pipeline with agent-driven updates, where every build is freshly and differently obfuscated. In that regime, analysing any one build may become economically silly, for three reasons.

One: a single decompile is already expensive. On our small corpus a best-in-class automated decompiler already spends tens of seconds per build at a moderate dial, and the breadth result shows that cost grows linearly with the number of obfuscated functions, so a real codebase would scale accordingly. If analysing one build costs hours and a new build ships daily, the analysis never catches its target.

Two: complementary methods may compound. Flattening, hardening and diversity are not the only transformations possible; we expect further, independent methods to emerge that raise the same cost, potentially multiplicatively, as hardening already multiplies flattening here. The curve we measure is super-linear in the dial; what we hope to reach, with further thought, is a transformation whose cost of decompilation grows *exponentially* in the dial while the binary's runtime stays flat.

Three: even a working exploit may not fire. Method C selects one of `K` variant bodies per call from a wall-clock hash. We measured this selection to be uniform over the `K` bodies: at `K=8` over a million calls each body received within half a percent of its `1/8` share, a chi-square goodness-of-fit consistent with uniform at `p=0.05`. So which variant's code executes on a given request is not predictable. An exploit whose reliability depends on a specific variant's code layout would therefore be reached only when that variant is selected, roughly `1/K` of the time, and so would not fire deterministically against a remote target. And by the time such an exploit is developed, a moving-target pipeline has recompiled the binary some number of times with fresh wirings, leaving a build-specific exploit stale. We are explicit that this argues only about layout-dependent exploitation: a bug in the shared logic or in input handling runs regardless of variant, and we have not built an exploit to test any of this. These claims should be validated empirically in further review.

8 Conclusion and reproducibility

Conspicuous compilation, with our simple and rather naive methods, raises automated-decompilation cost by up to nearly three orders of magnitude for behaviour that is byte-for-byte identical, and the cost structure is clean: methods A and B buy analysis-hostility for only a few nanoseconds per call, while method C trades a larger, scaling runtime cost for runtime non-determinism, which we validate directly: its per-call selector is empirically uniform over the `K` bodies (at `K=8` over 10^6 calls each body lands within half a percent of `1/K`, a chi-square consistent with uniform), and the opaque-predicate and MBA identities are proved and numerically checked (Appendix B). All code, seeds, pinned tool versions and the exact analysed binaries (with SHA-256) are in `experiments/` (decompile-cost, breadth-cost, variants-cost and runtime-cost), each with a build/run/aggregate pipeline and a reasoning FAQ, so every number regenerates from a single command (`experiments/rerun_experiments.sh`) at `github.com/ListeningPost/conspicuous-compilation`.

8.1 What this does not buy, and the defense it implies

The numbers above measure the cost to today's decompilers run naively: load, decompile, summarise. They do not stop a reasoning agent that defeats the transforms structurally. It is possible that these specific methods, in the configuration we provide, can be learned by the attacker, whether an agent or an agent swarm, and undone with new methodology. The goal of further research should be to find methods that combine with these, or supersede them, and that hold even against the best agents or swarms.

We strongly invite every reader, after enough time has passed to forget the approaches here, to go create a completely novel way to confuse and confound agents reasoning over binaries. It is going to take a village for this one.

APPENDICES

Appendix A The function corpus

The corpus is sixteen pure functions, each over an eight-word scratch buffer and kept out of the inliner so the decompiler must recover it individually. Two representative members are shown; the full source accompanies the artifact.

```
pub extern "C" fn even_fib_sum(buf: &mut [u64; 8],
_n: u64) -> u64 {
    buf[0] = 1; buf[1] = 2; buf[2] = 0;
    while buf[0] <= 4_000_000 {
        if (buf[0] & 1) == 0 { buf[2] =
buf[2].wrapping_add(buf[0]); }
        let t = buf[0].wrapping_add(buf[1]);
        buf[0] = buf[1]; buf[1] = t;
    }
    buf[2]
}
```

```
pub extern "C" fn hash_probe(buf: &mut [u64; 8], n:
u64) -> u64 {
    let mut h = 14695981039346656037u64 ^ n;
    for i in 0..8 {
        h ^= buf[i];
        h = h.wrapping_mul(1099511628211);
        h ^= h >> 23;
    }
    h
}
```

Obfuscation is one attribute macro: a function is protected by annotating it, and four knobs select the methods and their strength.

```
#[llnv_obfuscate(flatten=1, harden=1, jumps=64,
mba_depth=2, variants=4)]
pub extern "C" fn even_fib_sum(buf: &mut [u64; 8],
_n: u64) -> u64 { /* ... */ }
```

Notes for the reader. `flatten` toggles method A, `harden` method B, and `variants = K` method C; `jumps` sets the bogus-state count and `mba_depth` the masking depth. Functions are declared `#[no_mangle] extern "C"` so Rust keeps each as a standalone symbol. Every build evaluates an XOR-fold of all sixteen outputs, and that gate value is byte-identical across every condition, so a differing result aborts the build and each reported number is the cost of representing the same computation. Builds are reproducible with a pinned seed (`LLNV_SEED=7`); unset, the seed is drawn from the wall clock so each rebuild is wired differently.

Appendix B Mathematical formulation

Each step below is validated against the implementation; the two identities are proved and numerically checked. The Rust dispatcher keeps the real body as a single segment, so, unlike a multi-segment source flattener, there is no cross-segment def/use condition, and the corresponding equations are omitted deliberately.

Method A (flattening). States get distinct constants from a high band, and the dial fixes the bogus count, so the dispatcher is linear in it:

$$\sigma : \{\text{entry, real}\} \cup B \to [2^{20}, 2^{21}), \text{ injective} \tag{B1}$$

$$|B| = J, \quad M = |B| + 2 = J + 2 = \Theta(J) \tag{B2}$$

Method B (hardening). Each transition is guarded by an always-true opaque predicate, and state constants are MBA-masked by an always-zero term:

$$\Omega(a,b) = [\, 7a^2 - 1 \neq b^2 \,] = \text{true} \quad \text{for all } a,b \in \mathbb{Z} \tag{B3}$$

$$\mu(m) = m(m+1) \bmod 2 = 0, \quad v \mapsto v \oplus \mu(m) = v \tag{B4}$$

$$\text{st} \leftarrow \Omega(a,b) \;?\; \sigma(\text{real}) : \beta, \quad \beta \in B \tag{B5}$$

Proof of (B3). If $7a^2-1 = b^2$ then $b^2 \equiv -1 \equiv 6 \pmod 7$; the quadratic residues mod 7 are $\{0, 1, 2, 4\}$, and 6 is not among them, so no such b exists. Operands are masked to 30 bits, so $7a^2-1 \le 7(2^{30}-1)^2 < 2^{64}$ stays exact. **(B4)** holds because $m(m+1)$ is a product of consecutive integers and hence even, so the mask is identically zero; it still resists constant propagation because the operand passes an optimizer barrier. A brute-force check finds no counterexample to either identity up to $a = 3\times10^5$ and $m = 10^6$.

Method C (variants). One of K bodies is selected per call by the MurmurHash3 finalizer H over the wall-clock time t:

$$e = H(t) \bmod K \tag{B6}$$

Empirically e is uniform over $\{0, \ldots, K-1\}$: at K = 8, over 10^6 calls each body received 12.5% ± 0.1%, a chi-square goodness-of-fit consistent with uniform at p = 0.05 (median chi-square well below the 14.07 critical value for 7 d.o.f.). Consecutive calls repeat at a rate near the 1/K baseline, but that serial rate is a live-timing quantity sensitive to clock granularity and CPU state, so the reproducible selector statistic is the uniform body distribution (harness and artifact: `experiments/variants-cost/selector_uniformity.rs`). Each of the K bodies is drawn independently from the seeded generator with its own bogus-state count (around the dial), a randomized dead-edge permutation, and a shuffled arm order, so they are pairwise non-isomorphic control-flow graphs; for example at `jumps=12` four variants carried 9, 15, 14 and 23 bogus states at distinct layout positions.

Acknowledgments and disclosure

AI assistance. This work was produced with substantial AI assistance: a human-directed AI coding agent implemented the `#[llnv_obfuscate]` toolchain, the measurement harnesses,

and a draft of this manuscript, under the authors' direction. The authors reviewed the methods and results and take responsibility for the claims. We regard building defenses against agent-driven reverse engineering with agent assistance, in the open, as appropriate to the problem. We do not take our own numbers on faith, and we ask that you not either: the complete pipeline, pinned tool versions (rustc 1.94.1, Ghidra 12.1.4), the fixed seed (`LLNV_SEED=7`), and every analysed binary with its SHA-256 are published so that any reader, human or agent, can reproduce every figure independently on their own machine.

Code and data availability. All source, the build/run/aggregate pipelines, the raw Ghidra logs, and the aggregated results are available at `github.com/ListeningPost/conspicuous-compilation` under Apache-2.0. A single command, `experiments/rerun_experiments.sh`, regenerates every figure from clean builds, each run gated so a decompile that does not complete is never counted.

References

1. C. Collberg, C. Thomborson, and D. Low. A Taxonomy of Obfuscating Transformations. Tech. Rep. 148, Univ. of Auckland, 1997.
2. C. Collberg, C. Thomborson, and D. Low. Manufacturing Cheap, Resilient, and Stealthy Opaque Constructs. In *POPL*, pp. 184–196, 1998.
3. B. Barak et al. On the (Im)possibility of Obfuscating Programs. In *CRYPTO*, LNCS 2139, pp. 1–18, 2001.
4. P. Junod, J. Rinaldini, J. Wehrli, and J. Michielin. Obfuscator-LLVM — Software Protection for the Masses. In *IEEE/ACM SPRO*, pp. 3–9, 2015.
5. Y. Zhou, A. Main, Y. X. Gu, and H. Johnson. Information Hiding in Software with Mixed Boolean-Arithmetic Transforms. In *WISA*, LNCS 4867, pp. 61–75, 2007.
6. National Security Agency. Ghidra Software Reverse Engineering Framework. https://ghidra-sre.org and github.com/NationalSecurityAgency/ghidra. (Version 12.1.4, as used here.)
7. C. Wang, J. Hill, J. Knight, and J. Davidson. Software Tamper Resistance: Obstructing Static Analysis of Programs. Tech. Rep. CS-2000-12, Univ. of Virginia, 2000.
8. T. László and Á. Kiss. Obfuscating C++ Programs via Control Flow Flattening. *Annales Univ. Sci. Budapest., Sect. Comp.*, 30:3–19, 2009.